Ciências ULisboa | Informática

di.ciencias.ulisboa.pt

/difcul

Segurança e Confiabilidade

2025/2026

Trabalho 1 – 2ª fase

1 Objetivos

A parte prática da disciplina de Segurança e Confiabilidade pretende familiarizar os alunos com alguns dos problemas envolvidos na programação de aplicações distribuídas seguras, nomeadamente a gestão de chaves criptográficas, a geração de sínteses seguras, cifras e assinaturas digitais, e a utilização de canais seguros à base do protocolo TLS. O primeiro trabalho a desenvolver na disciplina será realizado utilizando a linguagem de programação Java e a API de segurança do Java, e é composto por duas fases.

O trabalho desenvolvido na primeira fase envolveu a implementação do ***Sperta***, um sistema de gestão de casas inteligentes (*smart homes*) com a arquitetura cliente-servidor mostrada na Fig.1. O programa cliente do ***Sperta***, denominado ***SpertaClient***, permite a um utilizador conectar-se ao servidor e gerir as suas casas inteligentes. Cada utilizador ligado ao servidor ***SpertaServer*** pode possuir uma ou mais casas e atribuir permissões a outros utilizadores para que estes possam gerir secções específicas de uma casa, ou mesmo a casa inteira. Todos os utilizadores com permissões para uma dada secção podem enviar ao servidor comandos para gerir os dispositivos da mesma (i.e., ligar/desligar ou ligar um dispositivo com temporização). Além disso, somente utilizadores com permissões numa dada secção podem obter informações sobre os dispositivos daquela secção, ou seja, informações sobre os comandos recebidos no servidor para ligar/desligar ou ligar um dispositivo com temporização.

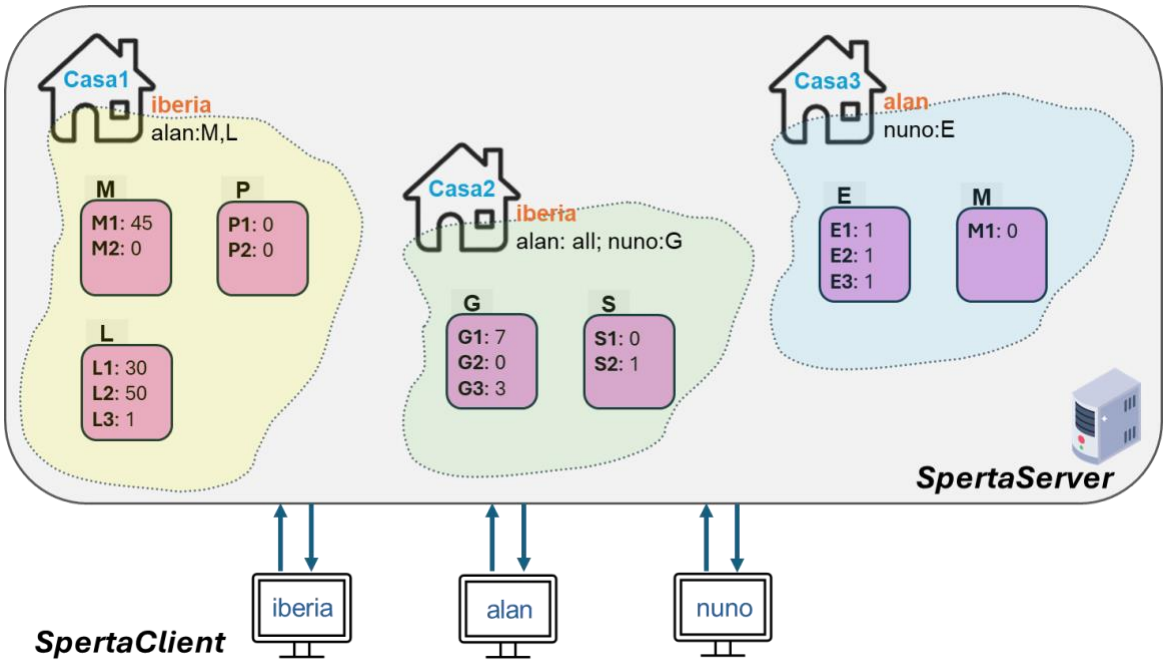

Fig.1. Diagrama da arquitetura do sistema ***Sperta***, com um servidor e múltiplos clientes.

A aplicação **servidor,** denominada ***SpertaServer,*** é um programa que permite a ligação com vários clientes em simultâneo, mantém informação sobre as casas, secções, dispositivos e utilizadores registados, autentica e identifica utilizadores, e vai colecionando e partilhando informação já recebida dos vários clientes, de forma organizada e persistente.

A primeira fase do projeto teve como objetivo fundamental a construção de uma aplicação distribuída, tendo sido concretizadas as funcionalidades oferecidas pelo serviço. Para isso, foram implementados os seguintes comandos:

- **CREATE <hm>** # Criar casa <hm> - utilizador é Owner
- **ADD <user1> <hm> <s>** # Adicionar utilizador <user1> à casa <hm>, seção <s>.
- **RD <hm> <s>** # **R**egistar um **D**ispositivo na casa <hm>, na seção <s>
- **EC <hm> <d> <int>** # **E**nviar valor <int> de estado/temporização, do dispositivo <d> da casa <hm>, para o servidor.
- **RT <hm>**# **R**eceber a informação sobre o último comando (estados/temporizações) enviado a cada dispositivo da casa <hm>, desde que o utilizador tenha permissões.
- **RH <hm> <d>**# **R**eceber o **H**istórico (ficheiro de log .csv) de comandos enviados ao dispositivo <d> da casa <hm>, desde que o utilizador tenha permissões.

Nesta segunda fase do projeto, serão considerados requisitos de segurança para que as interações e o sistema, no seu todo, sejam seguros. As várias funcionalidades definidas na primeira fase serão mantidas sem qualquer alteração (assim como a sintaxe dos comandos), embora, em alguns casos (nomeadamente nos comandos CREATE, ADD, EC, RT e RH), a sua concretização tenha de ser adaptada para satisfazer os requisitos de segurança.

2 Modelo de sistema e definições preliminares

A arquitetura do sistema segue o que foi definido na primeira fase do projeto. Contudo, agora **todas as comunicações serão feitas por meio de *sockets* TLS seguros, com autenticação unilateral, i.e., do servidor** .

As chaves privadas estarão armazenadas em ***keystores*** (uma por cada utilizador e uma para o servidor) protegidas por passwords. Para além disso, dado que todos os clientes terão necessidade de usar a chave pública do servidor para autenticá-lo, o certificado de chave pública (autoassinado – *self-sign certificate*) do servidor deve ser adicionado a uma ***truststore*** a ser usada pelo cliente, a qual também é protegida por uma password. Todos os pares de chaves públicas-privadas serão gerados com o algoritmo **RSA de 2048 bits**.

Para maximizar ainda mais a confiança no ambiente de execução, **o servidor tem de cifrar os ficheiros que contêm informações das casas, para além dos dados para atestação remota da aplicação cliente.** Isto garante que ninguém, para além do servidor, consegue ler estes ficheiros. A chave para cifrar os ficheiros deve ser gerada pelo algoritmo de PBE (*Password Based Encryption*) com AES de 128 bits e com base numa *password* indicada na inicialização do servidor, pela linha de comandos. É importante observar que o *salt* usado para gerar esta chave criptográfica deve ser guardado no servidor, de modo que seja o mesmo sempre que uma chave for gerada a partir da *password*, garantindo assim que a chave se mantém.

Para todos os ficheiros mantidos pelo servidor, será necessário assegurar que estes não foram corrompidos pela **verificação da sua integridade sempre** que se pretenda utilizar a informação neles contida. Neste sentido, cada ficheiro `<file>` terá a sua síntese calculada com base no algoritmo SHA256 e guardada num ficheiro de nome `<file>.hash` . Se a verificação de integridade falhar, seja na inicialização do servidor ***SpertaServer*** ou durante a sua execução, o servidor deve imprimir o aviso NOK-INTEGRITY e terminar imediatamente a sua execução. Note que, nos casos dos ficheiros que são cifrados pelo servidor (referidos no parágrafo anterior), a verificação de integridade será feita após os ficheiros serem decifrados. Deste modo, o uso de uma password incorreta na inicialização do servidor também resultará numa falha na verificação de integridade e, portanto, na emissão do aviso e no término do servidor.

Será ainda necessário **criar um mecanismo para atestação remota da aplicação cliente**. Este mecanismo deve ser seguro contra ataques do tipo *replay* (ver Seção 4.3).

Finalmente, será necessário **garantir confidencialidade fim-a-fim (*end-to-end*)** para todas as informações referentes aos comandos de alteração de estado dos dispositivos (ligar/desligar ou ligar um dispositivo com temporização) efetuados pelos clientes no âmbito de uma determinada secção. Isto significa que as informações destes comandos poderão ser decifradas apenas pelos utilizadores com permissões na secção a que o(s) dispositivo(s) indicado(s) nos comandos pertence(m). Utilizadores sem permissões na secção não poderão decifrar tais dados, assim como o servidor também não poderá.

Para usar o mecanismo de confidencialidade fim-a-fim, os utilizadores precisarão dos certificados das chaves públicas de outros utilizadores. Para viabilizar a partilha destes certificados entre utilizadores, o ***SpertaServer*** funcionará também como servidor de certificados de chaves públicas, ao qual o cliente ***SpertaClient*** recorrerá para obter certificados. Uma vez obtido um certificado de um utilizador, o cliente ***SpertaClient*** o guardará na sua ***truststore*** para uso futuro.

3 Utilização do Sistema Seguro

A execução das aplicações servidor e cliente deve ser feita da seguinte forma:

1. `SpertaServer <port> <password-cifra> <keystore> <password-keystore>`
    - `<port>` identifica o porto (TCP) para aceitar ligações de clientes. Por omissão, o servidor deve usar o porto 22345;
    - `<password-cifra>` é a password a ser usada para gerar a chave simétrica PBE que cifra os ficheiros da aplicação;
    - `<keystore>` que contém o par de chaves do servidor, incluindo o certificado da sua chave pública;
    - `<password-keystore>` é a password de acesso à *keystore*.
2. `SpertaClient <serverAddress> <truststore> <password-truststore> <keystore> <password-keystore> <user-id> <password>`
    - `<serverAddress>` identifica o servidor. O formato de serverAddress é o seguinte: `<IP/hostname>[:Port]`. O endereço IP ou *hostname* do servidor é obrigatório e o porto é opcional. Por omissão, o cliente deve ligar-se ao porto 22345 do servidor;
    - `<truststore>` que guardará os certificados de chave pública do servidor e dos demais utilizadores;
    - `<password-truststore>` é a password de acesso à *truststore*;

- `<keystore>` que contém o par de chaves do user-id, incluindo o certificado da sua chave pública;
- `<password-keystore>` é a password de acesso à *keystore*;
- `<user-id>` – string que identifica o utilizador no servidor ***SpertaServer***;
- `<password>` – string correspondente à password do utilizador.

4 Adicionar Segurança ao Sistema

4.1 Canais seguros TLS para comunicação segura e autenticação de servidores

Dado que na primeira fase do trabalho a comunicação entre cliente e servidor era realizada de forma insegura (canais em claro e sem autenticação do servidor), nesta segunda fase será necessário resolver este problema de segurança. Em concreto, será necessário garantir a **autenticidade do servidor** (um atacante não deve ser capaz de fingir ser o servidor e assim obter os dados do utilizador) e a **confidencialidade** da comunicação entre cliente e servidor (um atacante não deve ser capaz de escutar a comunicação). Para este efeito, devem-se usar **canais seguros** (protocolo TLS/SSL) e a verificação da identidade do servidor baseada em criptografia assimétrica (fornecida pelo protocolo TLS). Neste cenário, a autenticação é unilateral, somente do servidor; o cliente autentica-se da mesma forma que na fase 1 do projeto (i.e, user-id e password). Assim sendo, nesta fase do trabalho é então necessário:

- Utilizar ligações TLS: deve-se substituir a ligação TCP em claro por uma ligação TCP com TLS/SSL, uma vez que o protocolo TLS vai verificar a autenticidade do servidor e garantir a integridade e confidencialidade de toda a comunicação.
- Configurar as chaves necessárias: a utilização do protocolo TLS exige configurar as chaves tanto no cliente (*truststore* com o certificado auto-assinado do servidor) como no servidor (*keystore* com a sua chave privada e o certificado da sua chave pública).

4.2 Proteção e Autenticação de utilizadores

O servidor deve proteger a **confidencialidade das *passwords* dos utilizadores**. Com este objetivo, as passwords devem ser armazenadas utilizando mecanismos baseados em algoritmos de *síntese* e *salts*. O ficheiro dos utilizadores deve ser um **ficheiro de texto**, onde cada linha contém o *user-id*, o hash de (*password* || *salt*) e o *salt*, onde || corresponde à concatenação. O *hash* deverá ser calculado com o algoritmo SHA256.

```
Exemplo: <user-id>:<hash(password || salt)>:<salt>
alan:efnsuwiBuk+XDCu39plD6Wn6/kUnhWQZ7/eokKUJmUQ=:xQ+KUF46offqxaQ8pIz09Q==
```

Na primeira vez que um utilizador se autenticar no servidor, para além de registar `<user-id>:<hash(password || salt)>:<salt>`, o servidor deverá pedir ao ***SpertaClient*** o certificado do utilizador. O cliente ***SpertaClient*** deverá, então, enviar o certificado (ficheiro de formato **cert**) ao ***SpertaServer***, que o guardará para distribuí-lo conforme a necessidade no mecanismo de confidencialidade fim-a-fim descrito na Secção 4.4. Nas autenticações seguintes, o servidor deve recalcular o *hash* a partir da *password* fornecida e do *salt* armazenado, comparando-o com o *hash* guardado no ficheiro de utilizadores. Neste caso, o servidor não pedirá ao ***SpertaClient*** o certificado do utilizador.

4.3 Atestação Remota da Aplicação

O servidor deverá fazer a atestação remota da aplicação cliente. Na primeira fase, a atestação remota era feita apenas com base no tamanho do ficheiro executável ***SpertaClient***. Nesta segunda fase, a atestação remota será feita com base no *hash* SHA256 do ficheiro executável ***SpertaClient***.

Para que este mecanismo de atestação remota seja seguro contra ataques do tipo *replay*, é necessário implementar os seguintes passos:

1) Após a aplicação ***SpertaClient*** se ligar ao servidor, antes do utilizador fazer a sua autenticação, o servidor enviará ao cliente uma mensagem com um *nonce* aleatório de 8 bytes (por exemplo, um *long*).
2) Ao receber o *nonce*, o cliente o concatenará com o ficheiro executável ***SpertaClient***, calculará o *hash* SHA256 desta concatenação e enviará o *hash* calculado para o servidor.
3) Ao receber o *hash* calculado pelo cliente, o servidor o comparará com o *hash* calculado localmente com base na concatenação entre o mesmo *nonce* e uma cópia de referência da aplicação ***SpertaClient*** (i.e., o ficheiro `SpertaClient.jar`) guardada no servidor. Dois cenários podem ocorrer:
    a. O servidor não valida o programa cliente – o cliente recebe a mensagem do tipo NOK-ATTEST, a ligação é terminada, e o cliente termina assinalando o erro respetivo.
    b. O servidor valida o programa cliente – o cliente recebe a mensagem do tipo OK-ATTEST, e o processo segue em frente para a autenticação do utilizador.

Note que, no servidor, o ficheiro de texto que antes guardava o nome e o tamanho da aplicação ***SpertaClient***, agora não guardará o tamanho. No lugar deste, o ficheiro terá a diretoria e o nome da cópia de referência da aplicação ***SpertaClient*** guardada localmente.

4.4 Confidencialidade fim-a-fim para as mensagens

Na 1ª fase do projeto, os comandos de alteração de estado dos dispositivos (ligar/desligar e ligar com temporização) enviados por um utilizador podiam ser lidos pelo servidor e eram guardados por este em claro num ficheiro. Pretende-se agora nesta fase que seja assegurada confidencialidade fim-a-fim, ou seja, que apenas os utilizadores com permissões na secção à qual o dispositivo pertence possam ler estes dados. Para tal, os dados enviados ao servidor serão cifrados com criptografia simétrica (AES de 128 bits) utilizando uma **Chave da Secção**. Esta chave, criada pelo Owner da casa, será partilhada apenas com os demais utilizadores da secção por meio de criptografia assimétrica. Para isso, os seguintes pontos devem ser observados:

1) A **Chave da Secção** (simétrica) é criada pelo Owner da casa. Sempre que uma nova casa é criada, é gerada uma **Chave da Secção** para cada secção. Cada **Chave da Secção** é cifrada com a chave pública do Owner e, em seguida, enviada ao servidor, onde será armazenada com o nome `key.<hm>.<s>.<owner>`;
2) Cada vez que um Owner adicionar um utilizador a uma secção, este deverá enviar ao servidor a **Chave da Secção** cifrada com a chave pública do utilizador. Deste modo, ao fazer o comando `ADD <user> <hm> <s>`, deverá ocorrer o seguinte:
    a. Se o cliente ***SpertaClient*** já tiver o certificado do utilizador (`<user>`), ele deve pedir ao servidor ***SpertaServer*** a **Chave da Secção** da seção `<s>`, referente à casa `<hm>`, cifrada com sua chave pública, decifrá-la, cifrá-la com a chave pública do utilizador e enviá-la ao servidor para ser guardada com o nome `key.<hm>.<s>.<user>`;
    b. Se o cliente ***SpertaClient*** não tiver o certificado do utilizador, ele deverá pedir o certificado ao ***SpertaServer***. Ao receber o certificado, o cliente ***SpertaClient*** deverá executar o passo a), descrito acima.

3) Cada vez que um utilizador pedir dados de dispositivos (nomeadamente, informações de ligar/desligar ou ligar com temporização) por meio das operações `RT` e `RH`, o servidor deverá verificar se o utilizador tem permissão de acesso com base nas secções em que este se encontra registado. Se o utilizador tiver permissão de acesso, o servidor enviará os dados solicitados juntamente com a(s) respetiva(s) **Chave(s) da(s) Secção(ões)** (cifrada(s) com a chave pública do utilizador). Ao receber a(s) **Chave(s) da(s) Secção(ões)** cifrada(s), o ***SpertaClient*** a(s) decifrará usando a chave privada do utilizador. Em seguida, o ***SpertaClient*** decifrará os dados usando a(s) **Chave(s) da(s) Secção(ões)**. Se o utilizador não tiver permissão de acesso, o servidor enviará ao cliente ***SpertaClient*** uma mensagem NOPERM.
4) Sempre que um utilizador executar um comando `EC <hm> <d> <int>`, o cliente ***SpertaClient*** deverá enviar ao ***SpertaServer*** o valor `<int>` de estado/temporização cifrado com a **Chave da Secção** `<s>` da casa `<hm>` a que o dispositivo `<d>` pertence. Para concretizar esta operação, deverão ser observados os seguintes passos:
    a. O cliente ***SpertaClient*** envia ao servidor uma mensagem a informar que foi realizado um comando `EC` para o dispositivo `<d>` da casa `<hm>`;
    b. Se o utilizador tiver permissão na secção `<s>` da casa `<hm>` a que o dispositivo `<d>` pertence, o ***SpertaServer*** envia ao ***SpertaClient*** a respetiva **Chave da Secção** cifrada com chave pública do utilizador;
    c. O ***SpertaClient*** decifra a **Chave da Secção** `<s>` com a chave privada do utilizador e envia ao ***SpertaServer*** uma mensagem com o valor `<int>` cifrado com a **Chave da Secção** recebida.

5 Entrega

5.1 Trabalho

Dia **24 de abril**, até às 23:59 horas. O código desta segunda (e última) fase do trabalho deve ser entregue de acordo com as seguintes regras:

- A segunda fase do projeto deve ser realizada mantendo os grupos da primeira fase.
- Para entregar o trabalho, é necessário criar um **ficheiro zip** com o nome **SegC-grupoXX-proj1-fase2.zip**, onde **XX** é o número do grupo, contendo:
    - ✓ o código do trabalho;
    - ✓ conjunto de chaves, *keystores*, certificados (ficheiros de formato **cert**), *truststores*, e passwords de acesso às *keystores* e *truststores*;
    - ✓ os ficheiros jar (cliente e servidor) para execução do projeto;
    - ✓ um readme (txt) indicando **como compilar** e **como executar** o projeto, indicando também as limitações do trabalho.
- O ficheiro zip é submetido através da atividade disponibilizada para esse efeito na página da disciplina no Moodle. Apenas um dos elementos do grupo deve submeter. Se existirem várias submissões do mesmo grupo, será considerada a mais recente.
- <u>Não serão aceites trabalhos enviados por email</u>. Se não se verificar algum destes requisitos, o trabalho é considerado não entregue.

5.2 Autoavaliação de contribuições

Dia **25 de abril**, até às 23:59 horas. Cada aluno preenche no Moodle um formulário de autoavaliação das contribuições individuais de cada elemento do grupo na 2º fase do projeto. Por exemplo, se todos os elementos colaboraram de forma idêntica, bastará que todos indiquem que cada um contribuiu 33%. Aplicam-se as seguintes regras e penalizações:

- Alunos que não preencham o formulário **sofrem uma penalização de 10% na classificação.**
- Caso existam assimetrias entre as contribuições de cada elemento do grupo ou entre as autoavaliações de cada elemento do grupo, estas serão analisadas durante a discussão do trabalho e poderão levar à atribuição de notas individuais diferentes para cada elemento do grupo.